

David Nguyen

CSEN 296B-2, AI-Driven Software Development

Week 2.2 Lab: Backlog and Sprint Zero Plan

Everything here builds on my Week 1.2 requirements (FR-01 through FR-08, NFR-01 through NFR-03, SEC-01 through SEC-03) and the architecture I picked in Submission 1 (Next.js 14 + Supabase). My Working Agreement and Definition of Done from Week 2.1 still apply across all of this. Nothing counts as done unless it's testable, traces back to a requirement, and passes my DoD checklist.

User Stories and Acceptance Criteria

US-01: Create and save a note (from FR-01)

As a user, I want to create a new note with a title and Markdown body so that I can capture information quickly.

- A new note can be created from the main view with a single click/action.
- The note saves to Supabase Postgres on explicit save and persists across page reloads.
- An empty note (no title and no body) cannot be saved. The UI shows a warning.
- Markdown renders correctly in the editor preview (headings, bold, lists at minimum).

US-02: Edit an existing note (from FR-01)

As a user, I want to open and edit a previously saved note so that I can keep my information current.

- Clicking a note in the list opens it in the editor with existing content loaded.
- Changes persist on save and the updated timestamp reflects the edit.
- If the user navigates away with unsaved changes, a confirmation prompt appears.

US-03: Delete a note (from FR-01)

As a user, I want to delete a note I no longer need so that my workspace stays clean.

- A delete action is available from the note list or the open note view.
- A confirmation dialog appears before the note is permanently removed.
- After deletion the note no longer appears in the list or search results.

US-04: Organize notes into folders (from FR-02)

As a user, I want to organize my notes into folders so that I can group related content together.

- Users can create, rename, and delete folders from the sidebar.
- Every note belongs to exactly one folder (defaults to 'Uncategorized').
- Moving a note to a different folder updates the sidebar counts immediately.
- Deleting a folder that has notes in it prompts the user to move or delete them.

US-05: Search notes by keyword (from FR-03)

As a user, I want to search across all my notes by keyword so that I can find stuff without scrolling through everything.

- A search bar is accessible from the main view.
- Entering a keyword returns notes whose title or body matches.
- If nothing matches, a clear empty state message is shown.
- Results load within a reasonable time for up to a few hundred notes.

US-06: Sign up and log in (from SEC-01)

As a user, I want to create an account and log in so that my notes are private and only I can access them.

- User can sign up with email and password through Supabase Auth.
- User can log in and gets redirected to their notes dashboard.
- Unauthenticated users cannot access any notes or the main app view.
- Passwords are never stored in plaintext (Supabase Auth handles this).

US-07: Notes are private by default (from SEC-02, SEC-03)

As a user, I want my notes to be visible only to me so that sensitive information stays private.

- Row Level Security policies on the notes table restrict access to the owning user.
- A user cannot see, edit, or delete another user's notes even by manipulating API calls directly.
- RLS is enforced at the database level, not just in the UI.

US-08: Responsive layout (from NFR-02)

As a user, I want the app to work on both desktop and mobile browsers so I can access my notes from anywhere.

- The layout adjusts for screen widths from 375px (mobile) to 1440px+ (desktop).
- The sidebar collapses or turns into a hamburger menu on small screens.
- Core actions (create, edit, search) are usable on mobile without horizontal scrolling.

Prioritized Backlog

I ordered this by dependency and what gets me to a working vertical slice the fastest. Auth and basic CRUD have to come first because everything else depends on having a logged-in user who can create notes. Folders and search are next since those are the features that make the app actually useful day to day. Responsive layout is lower priority because it's more polish than functionality, and it doesn't block anything else.

High Priority

- US-06: Sign up and log in (everything requires auth)
- US-01: Create and save a note (core functionality, first vertical slice)
- US-02: Edit an existing note (completes the basic CRUD loop)
- US-03: Delete a note (finishes CRUD)
- US-07: Notes are private by default (RLS needs to be in place before real data hits the DB)

Medium Priority

- US-04: Organize notes into folders (first organizational feature)
- US-05: Search notes by keyword (usability at scale)

Low Priority

- US-08: Responsive layout (polish, doesn't block other stories)

Sprint Zero Plan

Sprint Zero is all prep, no feature code. The point is to make sure that when I actually start building US-06 and US-01, the project structure and tooling are already set up so I'm not wasting time figuring out config stuff mid-sprint.

Project setup: Initialize the Next.js 14 App Router project with TypeScript, Tailwind CSS, and shadcn/ui. Push to GitHub. Create the planning/ folder with requirements.md, user-stories.md, backlog.md, and sprint-zero-plan.md.

Supabase setup: Create the Supabase project. Set up the notes table schema (id, user_id, title, body, folder_id, created_at, updated_at). Enable Auth with email/password. Write the initial RLS policies.

Dev workflow: Make sure Copilot is working in VS Code. Set up .env.local for Supabase keys. Add a basic README with setup instructions so I can get back up and running if something breaks.

Technical decisions: Pick a Markdown editor library (leaning toward react-markdown with a textarea, or maybe Tiptap if I want something richer). Decide on the folder data model (separate folders table with a foreign key on notes vs. just a folder string field). Document these in the planning folder.

Risk reduction: Prototype the Supabase Auth login/signup flow in a throwaway page to make sure it works before building the real UI around it. Test that RLS actually blocks cross-user access with two test accounts.

Planning artifacts: Finalize the backlog and make sure every story traces back to at least one requirement from Week 1.2. Check everything against the DoD checklist from Week 2.1.

How AI Helped and What I Changed

I used Copilot Chat to get a first draft of user stories based on my Week 1.2 requirements. It gave me like 12 stories initially but a bunch of them were way too granular, like having separate stories for bolding text and italicizing text. Those aren't real slices of user value so I combined them into US-01 and trimmed the total down to 8.

The acceptance criteria Copilot suggested at first were pretty vague, stuff like 'the feature works as expected.' I asked it to rewrite them so they'd be specific enough to actually verify in a demo, which helped a lot. I still had to fix a few manually though, especially for US-07 where Copilot kept writing UI-level checks instead of database-level RLS enforcement.

Sprint Zero was mostly my own thinking. Copilot wanted to include things like setting up a full CI/CD pipeline and configuring monitoring, which would be nice but is way too much for a 10-week solo project right now. I kept it to the stuff that actually has to happen before I can start writing feature code: repo setup, Supabase config, an auth prototype, and documenting key technical decisions.